

free5GC + UERANSIM — End-to-End Test Guide

Deploy check, subscriber provisioning, configs, and the data-path test

coRAN Labs

June 2026

1. Overview

This guide walks the full end-to-end (e2e) test of **free5GC** (docker-compose) with a **native UERANSIM** gNB + UE: verify the core is healthy, **provision a subscriber** (free5GC does not auto-add one — the classic gotcha), point UERANSIM at the core, register the UE, bring up a PDU session, and prove the data path with a ping. Every value used here is the one validated on this VM; addresses are environment-specific (read them live).

Result achieved: UE registered, PDU session up (UE IP 10.60.0.1), and ping 8.8.8.8 -> 4/4 replies, ~5 ms, through the free5GC **gtp5g** UPF.

UERANSIM (native, on host)		free5GC (docker-compose)
-----	N2/NGAP (SCTP 38412)	-----
nr-gnb (10.100.200.1)	----->	amf 10.100.200.16
nr-ue -> uesimtun0	N3/GTP-U (2152)	smf 10.100.200.6
10.60.0.1	----->	upf 10.100.200.2 (gtp5g)
-----		-----
all on docker bridge br-free5gc 10.100.200.0/24		

2. Prerequisites

- **free5GC** deployed via docker-compose at ~/free5gc-compose (this used webui v4.2.3).
- **gtp5g** kernel module loaded on the host — free5GC's UPF datapath needs it:

```
lsmod | grep gtp5g      # must be present; else: cd ~/gtp5g && make && sudo make install && sudo modprobe gtp5g
```

- Native UERANSIM built at ~/UERANSIM (build/nr-gnb, build/nr-ue).
- Run gNB/UE with sudo (they create the uesimtun0 TUN + raw sockets).

3. Step 1 — Verify the core is healthy

```
cd ~/free5gc-compose
sudo docker ps --format '{{.Names}} | {{.Status}}'
```

Expect all NFs Up: amf smf nrf udm udr ausf nssf pcg upf n3iwf webui mongodb. Confirm the UPF datapath module:

```
lsmod | grep gtp5g      # gtp5g present
sudo docker exec upf ip -br addr | grep -vE 'lo'  # eth0 (N3) + upfgtp (the gtp5g tun)
```

4. Step 2 — Read the live addresses (environment-specific)

```
ip -br addr | grep 10.100.200      # host IP on the free5gc bridge (here 10.100.200.1)
for c in amf smf upf; do echo -n "$c: "; \
```

```
sudo docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}} {{end}}' $c; done
```

On this VM: | Element | Address | |—| | free5gc bridge (host side) | br-free5gc 10.100.200.1/24 | | AMF (N2/NGAP target) | 10.100.200.16 | | SMF | 10.100.200.6 | | UPF (N3 / gtp5g) | 10.100.200.2 | | UE pool (assigned) | 10.60.0.0/16 -> UE got 10.60.0.1 |

5. Step 3 — Provision the subscriber (the free5GC gotcha)

free5GC ships **no subscribers** — the UE cannot register until you add one matching the UE config. Check first:

```
sudo docker exec mongod mongo --quiet free5gc \
--eval 'db.getCollection("subscriptionData.authenticationSubscription").count()'
# 0 -> none provisioned
```

The UE identity/credentials we provision (must match free5gc-custom-ue.yaml): | Field | Value | |—| | SUPI / IMSI | imsi-208930000000001 (MCC 208, MNC 93) | | Key (K) | 8baf473f2f8fd09487cccbd7097c6862 | | OPC | 8e27b6af0e692e750f32667a3b14605d (opType **OPC**) | | AMF (auth mgmt field) | 8000 | | S-NSSAI | SST 1, SD 010203 | | DNN | internet |

Provision via the WebUI REST API (port 5000; login admin / free5gc). Save as /tmp/free5gc_prov.py and run python3 /tmp/free5gc_prov.py:

```
import json, urllib.request
BASE="http://localhost:5000"
UE="imsi-208930000000001"; PLMN="20893"
K="8baf473f2f8fd09487cccbd7097c6862"; OPC="8e27b6af0e692e750f32667a3b14605d"
AMF="8000"; SST=1; SD="010203"; DNN="internet"; SQN="000000000000"

def req(method, path, body=None, token=None):
    data=json.dumps(body).encode() if body is not None else None
    r=urllib.request.Request(BASE+path, data=data, method=method)
    r.add_header("Content-Type","application/json")
    if token: r.add_header("Token", token)
    try:
        with urllib.request.urlopen(r, timeout=15) as resp: return resp.status, resp.read().decode()
    except urllib.error.HTTPError as e: return e.code, e.read().decode()

st,out=req("POST","/api/login,{"username":"admin","password":"free5gc"}")
tok=json.loads(out)["access_token"]; print("login:",st)

sub={
    "plmnID":PLMN,"ueId":UE,
    "AuthenticationSubscription":{
        "authenticationManagementField":AMF,"authenticationMethod":"5G_AKA",
        "mileneage":{"op":{"encryptionAlgorithm":0,"encryptionKey":0,"opValue":""}},
        "opc":{"encryptionAlgorithm":0,"encryptionKey":0,"opcValue":OPC},
        "permanentKey":{"encryptionAlgorithm":0,"encryptionKey":0,"permanentKeyValue":K},
        "sequenceNumber":SQN },
    "AccessAndMobilitySubscriptionData":{
        "gpsis":["msisdn-09000000000"],
        "nssai":{"defaultSingleNssai":[{"sst":SST,"sd":SD}],"singleNssais":[{"sst":SST,"sd":SD}]},
        "subscribedUeAmbr":{"downlink":"2 Gbps","uplink":"1 Gbps"}},
    "SessionManagementSubscriptionData":[{"
        "singleNssai":{"sst":SST,"sd":SD},
        "dnnConfigurations":{"DNN:{
            "pduSessionTypes":{"defaultSessionType":"IPv4","allowedSessionTypes":["IPv4"]},
            "sscModes":{"defaultSscMode":"SSC_MODE_1","allowedSscModes":["SSC_MODE_1"]},
            "5gQosProfile":{"5qi":9,"arp":{"priorityLevel":8,"preemptCap":"","preemptVuln":""},"priorityLevel":8}}

```

```

    "sessionAmbr":{"downlink":"1000 Mbps","uplink":"1000 Mbps"}}}],
"SmfSelectionSubscriptionData":{"subscribedSnssaiInfos":{"f":{"SST:02x}{SD}":{"dnnInfos":[{"dnn":DNN}]}}},
"AmPolicyData":{"subscCats":["free5gc"]},
"SmPolicyData":{"smPolicySnssaiData":{"f":{"SST:02x}{SD}":{"snssai":{"sst":SST,"sd":SD},
    "smPolicyDnnData":{"DNN":{"dnn":DNN}}}}},
"FlowRules":[],"QosFlows":[],"ChargingDatas":[]
}
st,out=req("POST",f"/api/subscriber/{UE}/{PLMN}",sub,tok); print("provision:",st)
st,out=req("GET",f"/api/subscriber/{UE}/{PLMN}",None,tok); print("verify GET:",st)

```

Expect login: 200, provision: 201, verify GET: 200. Re-check mongo -> count 1.

Alternative: the WebUI at <http://<host>:5000> (login admin/free5gc) -> “Subscribers” -> “New Subscriber” with the same values. The API is just faster/repeatable.

6. Step 4 — UERANSIM configs (separate custom files)

Keep the stock free5gc-gnb.yaml/free5gc-ue.yaml (which use 127.0.0.1) **untouched**; create **custom** copies pointed at the live bridge:

```

cd ~/UERANSIM/config
cp free5gc-gnb.yaml free5gc-custom-gnb.yaml
cp free5gc-ue.yaml free5gc-custom-ue.yaml

```

Edits in **free5gc-custom-gnb.yaml** (point the gNB at the bridge + AMF):

```

linkIp: 10.100.200.1      # RLS toward the UE (UE gnbSearchList must match this)
ngapIp: 10.100.200.1     # N2 toward AMF (host IP on br-free5gc)
gtpIp: 10.100.200.1      # N3 GTP-U source
amfConfigs:
  - address: 10.100.200.16 # AMF NGAP (was 127.0.0.1)
    port: 38412
# mcc '208' / mnc '93', slice sst 1 sd 0x010203 - already correct

```

Edit in **free5gc-custom-ue.yaml** (must match the gNB's linkIp):

```

gnbSearchList:
  - 10.100.200.1          # was 127.0.0.1
# supi imsi-208930000000001, key/op as provisioned, apn internet, sst 1 sd 0x010203

```

7. Step 5 — Run the gNB and UE

```

cd ~/UERANSIM
sudo pkill -x nr-gnb; sudo pkill -x nr-ue          # clean any prior run
sudo ./build/nr-gnb -c config/free5gc-custom-gnb.yaml >/tmp/f5_gnb.log 2>&1 &
sleep 5; grep -i "NG Setup" /tmp/f5_gnb.log        # expect "NG Setup procedure is successful"
sudo ./build/nr-ue -c config/free5gc-custom-ue.yaml >/tmp/f5_ue.log 2>&1 &
sleep 8; grep -iE "Registration|PDU Session|uesimtun" /tmp/f5_ue.log

```

Expected: Initial Registration is successful -> PDU Session establishment is successful PSI[1] -> TUN interface[uesimtun0, 10.60.0.1] is up.

8. Step 6 — Verify the data path (through the gtp5g UPF)

```

sudo ping -I uesimtun0 -c4 8.8.8.8

```

Validated result: 4/4 received, 0% loss, ~5 ms. (If uesimtun0 is in a network namespace, prefix with `sudo ip netns exec <ns> ping ....`)

9. Problems solved (and why)

Symptom	Cause	Fix
UE never registers / no subscriber	free5GC has no subscriber by default (count 0)	Provision via WebUI API (Step 3)
PLMN/Cell selection failure, no cells in coverage	UE <code>gnbSearchList</code> (127.0.0.1) gNB <code>linkIp</code>	Set <code>gnbSearchList:</code> 10.100.200.1
gNB SCTP Connection refused to AMF	gNB config pointed at 127.0.0.1 not the AMF container	<code>amfConfigs.address:</code> 10.100.200.16
Authentication Failure due to SQN out of range	first-attach SQN resync (normal)	none — it auto-resyncs and registration succeeds

10. Notes

- **free5GC UPF uses the gtp5g kernel module** (a real GTP-U netdev), so it forwards fine over Docker bridge veths — unlike OAI's eBPF/XDP datapath, which puts a bridge veth into NO-CARRIER on this VM. That's why free5GC e2e works here out of the box.
- **Subscriber persistence:** the subscriber lives in free5GC's `mongodb`. It survives core restarts as long as the mongo volume persists; a full `docker compose down -v` wipes it -> re-run Step 3.
- **Addresses are per-deployment.** If the `br-free5gc` subnet / AMF IP differ on another host, re-read them (Step 2) and update the custom gNB/UE files accordingly.
- **upfbench:** free5GC is a strong candidate as a 4th UPF target. Unlike Open5GS (whose UPF only programs `gtp5g` for its own SMF), free5GC's UPF may program `gtp5g` for an external `pfcp`sim-driven session — worth testing whether Suites 1/2 forward before committing an adapter.